

```
$ onboard jordanvalnet/exalthon26 --profile dev
```

Onboard, pipeline d'agents Claude Code sur le serveur MCP GitHub : né le 9 septembre 2026, sans CI ni licence, mais assez balisé pour commiter dès la première heure

Hackathon eXaltemps 2026 : agent + MCP GitHub

0

ÉTOILES GITHUB

5

CONTRIBUTEURS ACTIFS
sur 12 semaines

0

PR OUVERTES

10 sept. 2026

DERNIER COMMIT

<https://github.com/jordanvalnet/exalthon26> Sans licence Onboard · profil dev

Que fait ce projet et pour qui ?

> Un CLI `bun onboard owner/repo profil` qui fait tourner des agents Claude Code sur le serveur MCP GitHub pour produire un deck sourcé par public ¹ ; c'est un livrable de hackathon, sans licence à ce jour ².

Onboard parcourt n'importe quel dépôt GitHub par le serveur MCP GitHub, construit un cache documentaire vérifié et en tire une présentation par public : dev, qa, cto, ceo, investisseur, enfant ; chaque chiffre renvoie à sa source et un guide répond ensuite aux questions à partir du cache ¹.

La commande unique est `bun onboard owner/repo profil` : elle lance Claude Code en non interactif avec le skill `/onboard` ; dans une session Claude Code, les mêmes étapes sont des skills (`/onboard` , `/onboard-collect` , `/onboard-write` , `/onboard-render` , `/onboard-ask`) et les prompts vivent dans `onboard/agents/` , exécutables aussi par Copilot et Codex ¹.

Fiche GitHub : `jordanvalnet/exalthon26` , « Hackathon eXaltemps 2026 : agent + MCP GitHub » ³ , compte personnel ⁴ , branche `main` ⁵ , créé le 2026-09-09 ⁶ , 0 star et 0 fork ^{7 8} , aucun topic ⁹ , aucun fichier LICENSE ².

Pour qui : le lecteur final est l'un des six publics ; vous, vous rejoignez le livrable de l'équipe eXaltemps au hackathon « Agent + MCP GitHub » du 9 septembre 2026, consignes du jury dans `HACKATHON.md` , dépôts cibles dans `CIBLES.md` ^{1 10 11}.

Contrainte d'architecture à connaître avant d'écrire une ligne : GitHub n'est joint que par le serveur MCP `github` déclaré dans `.mcp.json` (`https://api.githubcopilot.com/mcp/` , en-tête `Bearer ${GITHUB_PAT}`), depuis l'agent comme depuis `src/github.ts` , jamais par appel REST direct ¹².

Comment le code est-il organisé ?

> Tout le code est dans `src/` (9 entrées), tout le pipeline dans `onboard/` : prompts, profils, contrats, caches ^{13 14}.

- └ `.claude/` skills Claude Code du pipeline
 - └ `chat/` client du chat d'équipe, chat.mjs
 - └ `docs/` documentation
 - └ `onboard/` pipeline Onboard : agents, profils, cache, contrats
 - └ `onboarding/` second package bun, collecteurs dans src
 - └ `pitch/` supports du pitch
 - └ `src/` TypeScript : CLI, collecte, rendu, validation, fusion
 - └ `test/` 2 tests bun
 - └ `.env.example` modèle de variables d'environnement
 - └ `.gitignore` exclusions git
 - └ `.mcp.json` config serveur MCP GitHub
 - └ `AGENTS.md` consignes pour agents IA
 - └ `CIBLES.md` repos cibles du hackathon
 - └ `CLAUDE.md` instructions Claude Code, 69 octets
 - └ `HACKATHON.md` consignes du hackathon
 - └ `LIVRABLE.md` rapport de mission livré
- ... et 5 autres entrées, voir `sources/tree.md`

Racine de 21 entrées, sans dossier `.github/` ^{13 15}. Deux mondes : le code TypeScript dans `src/` (CLI, collecte, rendu, validation, fusion) ¹⁶ et le pipeline documentaire dans `onboard/` (agents, profils, cache, contrats) ¹⁴.

CHEMIN	CONTENU	SOURCE
<code>src/</code>	cli.ts, collect/, facts.ts, gh.ts, github.ts, index.ts, merge.ts, render/, validate.ts	¹³
<code>onboard/</code>	PLAN.md, README.md, SCHEMA.md, WORKFLOW.md, agents/, cache/, profiles/	¹³
<code>onboarding/</code>	second package bun, collecteurs dans src	¹⁷
<code>.claude/</code>	skills Claude Code du pipeline	¹⁸
<code>chat/</code>	client du chat d'équipe, chat.mjs	¹⁹
<code>test/</code>	2 tests bun	²⁰
<code>docs/</code> , <code>pitch/</code> , <code>pitch_fr.html</code>	documentation, supports du pitch, pitch HTML en français	^{21 22 23}

Deux `package.json` : celui de la racine, 15 scripts, une dépendance runtime (zod) et deux de dev (@types/bun, typescript) ²⁴ ; celui d' `onboarding/` , 379 octets, non lu par la collecte ²⁵.

Le repo est balisé pour les agents : `AGENTS.md` , `CLAUDE.md` , `.mcp.json` , `.claude/skills/` , soit 4 repères sur 7 ^{26 27} ; manquent `.cursorrules` , `.github/copilot-instructions.md` et `llms.txt` ¹².

La cache ne contient pas le contenu de `.claude/` , `chat/` , `docs/` , `pitch/` , `src/collect/` , `src/render/` , `onboarding/src/` , `onboard/agents/` , `onboard/cache/` , `onboard/profiles/` : non descendus par la collecte, à parcourir dans le repo ¹³.

Par où commencer à lire ?

> Ouvrez `src/cli.ts` puis `onboard/agents/0-onboard.md` : le CLI n'est qu'un lanceur, la logique est dans les prompts d'agents ^{28 1}.

Ordre de lecture prescrit par `AGENTS.md` : `onboard/README.md` (schéma du flux), `onboard/WORKFLOW.md`, `onboard/SCHEMA.md`, `onboard/PLAN.md` (rôles), `onboard/agents/`, `onboard/profiles/` ¹². Le README de la racine en donne la version courte : quatre étapes, collecter, rédiger, rendre, guider, chacune ne lisant que les sorties de la précédente et ne démarrant que si la validation en code passe ¹.

Puis le code, dans cet ordre :

1. `src/cli.ts` : script `onboard` de `package.json`, `bun onboard owner/repo profil [--force]` ; vérifie les arguments, le profil (`readProfile` de `./validate.ts`), `GITHUB_PAT` et la présence de `claude` dans le PATH, puis lance `onboard/agents/0-onboard.md` via Claude Code ; `cache = onboard/cache/<owner>__<repo>` ²⁸.
2. `src/validate.ts` : script `validate`, exporte `readProfile`, le plus gros fichier de `src/` (8324 octets) ²⁹ ; à lire avec `src/facts.ts` (schéma zod du cache) et `src/merge.ts` (fusion des collectes) ¹.
3. `src/render/index.ts` : script `render`, produit le deck HTML depuis le cache ; voisins `pdf.ts` et `screenshot.ts` ³⁰ ; `theme.ts` une identité par profil, `kpi.ts` tuiles, `charts.ts` SVG, `deck.ts` assemblage ¹.
4. `src/index.ts` : scripts `dev` et `start`, appelle `whoami()` de `./github.ts`, affiche « GitHub OK : <login> » ou sort en code 1 ³¹.
5. `chat/chat.mjs` : script `chat`, client du chat d'équipe, seul `.mjs` cité par les scripts ³².

Honnêteté du cache : seuls `src/cli.ts` (40 premières lignes) et `src/index.ts` ont été lus ; `src/render/index.ts`, `src/validate.ts` et `chat/chat.mjs` sont déduits du manifeste, à confirmer en ouvrant les fichiers ^{28 30 29}.

05

Comment builder, tester et lancer ?

> Aucune CI : `bun run check` (`tsc --noEmit && bun test`) avant chaque push est la seule barrière ^{33 1}.

```
curl -fsSL https://bun.sh/install | bash      # Windows : powershell -c "irm bun.sh/install.ps1 | iex"
cp .env.example .env                        # GITHUB_PAT=<token GitHub, scope repo>
bun install && bun run check                  # tsc --noEmit && bun test
bun onboard owner/repo dev                  # collecte, rédaction, rendu : quelques minutes
bun run pdf onboard/cache/<owner>__<repo> dev # le PDF, via Chrome ou Edge en headless
```

Les commandes viennent du README ¹ ; installer = `bun install` , lancer = `bun onboard owner/repo profil` , tester = `bun run check` ^{34 35 36}.

Runtime bun, jamais npm, TypeScript strict ; bun charge `.env` tout seul, `process.env.GITHUB_PAT` est disponible ¹². Pour que le serveur MCP voie le token dans une session Claude Code : `set -a; source .env; set +a` avant de lancer l'assistant, puis `/mcp` pour vérifier que `github` est connecté ¹.

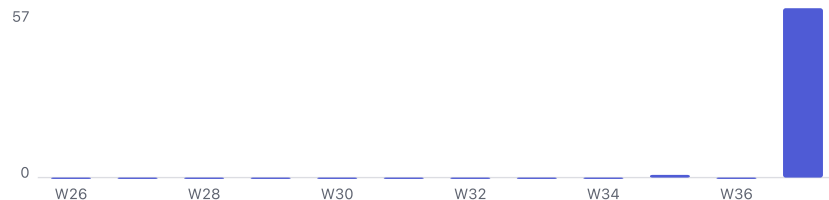
Tests : `bun test` , 2 fichiers dans `test/` (`github.test.ts` , `issues.test.ts`), ni vitest ni jest ^{37 38 39} ; `check` = `bun run typecheck && bun test` , soit `tsc --noEmit` puis `bun test` ²⁴.

CI : aucune. Pas de `.github/workflows/` , `build.ci` est vide, la seule vérification est locale ^{40 41 33}. Dépendances : 3 déclarées, 1 runtime (zod ^4.5.4), `bun.lock` versionné ^{42 43 44} ²⁵.

Autres scripts : `render` , `validate facts|narrative|deck` , `merge` , `meta` , `issues` , `gh` , `screenshot` , `pdf` ; 15 au total ²⁴. Le cache ne contient pas `sources/contributing.md` : il n'y a pas de CONTRIBUTING.md à la racine, les règles de travail sont dans `AGENTS.md` ^{13 12}.

Comment le projet vit-il ?

> Un sprint de hackathon : 57 commits en une semaine, cinq comptes, jordanvalnet en porte 41, bus factor 1 [45](#) [46](#) [47](#) [48](#).



Le repo a été créé le 2026-09-09T13:24:36Z et le dernier commit date du 2026-09-10T08:37:39Z : il a l'âge du hackathon [6](#) [49](#).

Sur 12 semaines, 57 commits en 2026-W37, 1 commit daté 2026-W35, 0 partout ailleurs [45](#) [50](#) [51](#).

COMPTE	COMMITTS	SOURCE
jordanvalnet	41	47
PrincyExaltIT	8	52
Sacane	4	53
radomd92	3	54
loic.vyncke	2	55

Bus factor 1 : un seul contributeur porte la moitié des commits des 12 semaines [48](#) [56](#). Aucune release [57](#) ; version `0.1.0` dans package.json [24](#).

Flux de travail : 0 PR ouverte [58](#) et 1 PR fusionnée le 2026-09-09T14:16:45Z, #2 « Onboard : workflow agentique, contrat du cache, profils, agents, validations » [59](#) [60](#), ce qui indique des pushes directs sur `main` pour le reste des commits [5](#).

L'équipe coordonne ses assistants par l'issue #1, 22 commentaires : un commentaire = un message, `node chat/chat.mjs read|send|who|wait` [61](#) [12](#).

Quelle première contribution ?

> Pas de good first issue : la contribution la plus utile est un workflow CI qui lance `bun run check` , aujourd'hui absent ^{62 33} .

Pas de piste balisée : 1 issue ouverte, 0 fermée sur 30 jours, aucun label, aucune « good first issue » ^{63 64 65 62} .

L'unique issue ouverte, #1 «  Chat équipe — canal IA ↔ IA », 22 commentaires, n'est pas une demande : c'est le canal de chat des assistants de l'équipe, un commentaire = un message, à ne jamais fermer ^{66 61 67 12} .

Côté PR : 0 ouverte ⁵⁸ , 1 fusionnée, #2, le 2026-09-09 ^{68 60} . Le cache ne contient pas `sources/contributing.md` (pas de CONTRIBUTING.md à la racine) ; les règles connues sont celles d' `AGENTS.md` : bun jamais npm, GitHub uniquement par le serveur MCP, `bun run check` avant de rendre la main ^{13 12} .

Les manques relevés par la collecte font de bonnes premières contributions, petites et vérifiables :

PISTE	POURQUOI	SOURCE
Workflow GitHub Actions qui lance <code>bun run check</code>	aucun workflow, vérification locale seulement	³³
Fichier <code>LICENSE</code>	réutilisation juridiquement incertaine	²
<code>SECURITY.md</code>	aucun canal déclaré pour signaler une faille	⁶⁹
Tests pour <code>merge</code> , <code>validate</code> , <code>render</code>	2 tests seulement, <code>github</code> et <code>issues</code>	^{38 39}
<code>llms.txt</code> , <code>.github/copilot-instructions.md</code>	4 repères IA sur 7	^{26 12}

Aucun `TODO` de dette technique dans le code : les 10 résultats de la recherche sont des artefacts du pipeline ou un identifiant dans `onboarding/src/report/skeleton.ts` ; `FIXME` n'a pas été cherché ⁷⁰ .

\$ Vos 3 premières actions

> Terminal ouvert, `bun run check` vert, un premier commit utile dès la première heure : le workflow CI qui manque ³³.

1. Installer et vérifier l'accès GitHub : `cp .env.example .env` (GITHUB_PAT, scope repo), `bun install` && `bun run check`, puis `bun run dev` qui doit afficher « GitHub OK : <login> » ^{1 34 31}.
2. Lire `onboard/README.md`, `onboard/WORKFLOW.md`, `onboard/SCHEMA.md`, `onboard/PLAN.md`, puis `src/cli.ts` et `onboard/agents/0-onboard.md`, et lancer `bun onboard owner/repo dev` sur un dépôt de `CIBLES.md` pour voir le cache se remplir dans `onboard/cache/<owner>__<repo>/` ^{12 28 11}.
3. Se signaler sur le chat d'équipe, `node chat/chat.mjs send "👋 ..."` (issue #1) ^{12 67}, puis proposer le workflow GitHub Actions qui lance `bun run check` : premier manque relevé par la collecte, sans risque pour le pipeline ^{33 36}.

NOTES

Sources

1. source du repo `readme.md`
2. donnée collectée `risks.0.note`
3. donnée collectée `repo.description`
4. donnée collectée `repo.owner_type`
5. donnée collectée `repo.default_branch`
6. donnée collectée `repo.created_at`
7. donnée collectée `repo.stars`
8. donnée collectée `repo.forks`
9. donnée collectée `repo.topics`
10. donnée collectée `tree.7.role`
11. donnée collectée `tree.5.role`
12. source du repo `ai-docs.md`
13. source du repo `tree.md`
14. donnée collectée `tree.13.role`
15. donnée collectée `tree`
16. donnée collectée `tree.18.role`
17. donnée collectée `tree.14.role`
18. donnée collectée `tree.0.role`
19. donnée collectée `tree.11.role`
20. donnée collectée `tree.19.role`
21. donnée collectée `tree.12.role`
22. donnée collectée `tree.16.role`
23. donnée collectée `tree.17.role`
24. source du repo `manifest.md`
25. donnée collectée `risks.4.note`
26. donnée collectée `ai_docs.score`
27. donnée collectée `ai_docs.max`
28. donnée collectée `entrypoints.0.why`
29. donnée collectée `entrypoints.3.why`
30. donnée collectée `entrypoints.2.why`
31. donnée collectée `entrypoints.1.why`
32. donnée collectée `entrypoints.4.why`
33. donnée collectée `risks.3.note`
34. donnée collectée `build.install`
35. donnée collectée `build.run`
36. donnée collectée `build.test`
37. donnée collectée `tests.framework`
38. donnée collectée `tests.files`
39. source du repo `tests.md`
40. donnée collectée `build.ci`
41. source du repo `ci.md`
42. donnée collectée `deps.count`
43. donnée collectée `deps.runtime.0.name`
44. donnée collectée `deps.runtime.0.version`
45. donnée collectée `activity.commits_per_week.11.count`
46. donnée collectée `activity.contributors`
47. donnée collectée `activity.contributors.0.commits`
48. donnée collectée `activity.bus_factor`
49. donnée collectée `activity.last_commit`
50. donnée collectée `activity.commits_per_week.9.count`
51. donnée collectée `activity.commits_per_week`
52. donnée collectée `activity.contributors.1.commits`
53. donnée collectée `activity.contributors.2.commits`
54. donnée collectée `activity.contributors.3.commits`
55. donnée collectée `activity.contributors.4.commits`
56. donnée collectée `risks.2.note`
57. donnée collectée `releases`
58. donnée collectée `pulls.open`
59. donnée collectée `pulls.merged_30d.0.title`
60. donnée collectée `pulls.merged_30d.0.merged_at`
61. donnée collectée `issues.hot.0.comments`
62. donnée collectée `issues.good_first`
63. donnée collectée `issues.open`
64. donnée collectée `issues.closed_30d`
65. donnée collectée `issues.by_label`
66. donnée collectée `issues.hot.0.title`
67. donnée collectée `issues.hot.0.url`
68. donnée collectée `pulls.merged_30d.0.number`
69. donnée collectée `risks.1.note`
70. source du repo `todo.md`